

VCMalloc: A Virtually Contiguous Memory Allocator

Yacine Hadjadj , Chakib Mustapha Anouar Zouaoui , Nasreddine Taleb , *Senior Member, IEEE*,
Sarah Mazari , Mohamed El Bahri , and Miloud Chikr El Mezouar , *Senior Member, IEEE*

Abstract—This paper presents VCMalloc, a custom memory allocator based on a new dynamic memory management approach that keeps data allocated in a contiguous form in memory. It can preserve the virtual contiguity of data even after performing different memory operations like “Reallocation” and “Free” without consuming much system resources. A novel control plane that we call a “Memory Management System” is introduced with a set of algorithms for the physical-virtual memory mappings, assuring fast, efficient, and safe tracking of data and user pointers. The proposed allocator prioritizes virtual contiguity while it keeps the physical memory completely up to the operating system to manage. It was tested and compared with the current implementation of Malloc as well as a state-of-the-art memory allocator MIMalloc in the recent versions of the Microsoft Windows operating system. For the allocator itself, a considerable performance increase has been achieved in basic operations of up to 28% increase in allocation speed and 26% in reallocation speed. The data structures allocated by VCMalloc have shown remarkable performance increases of up to 31% faster for matrix multiplication. Furthermore, VCMalloc has also demonstrated its superiority in real-world SPEC CPU 2017 benchmark applications.

Index Terms—Contiguous memory, memory allocator, paging, cache awareness, data structures.

I. INTRODUCTION

MEMORY allocation is a crucial procedure in which available memory is assigned to programs. This allocation can be either static or dynamic. Static memory allocation occurs at compile time and involves assigning a fixed memory area known in advance. This type of allocation takes place at the stack memory level and lacks the ability to release or resize memory during run-time. On the other hand, dynamic memory allocation takes place in the HEAP memory and accommodates sizes that are determined at run-time. This approach allows memory to be allocated on demand and facilitates the release and resizing operations based on the specific requirements of the application. However, dynamic memory management is a

Manuscript received 19 June 2022; revised 16 June 2023; accepted 27 July 2023. Date of publication 7 August 2023; date of current version 8 November 2023. This work was supported by the Algerian Directorate General for Scientific Research and Technological Development (DGRSDT). Recommended for acceptance by C. Li. (*Corresponding author: Yacine Hadjadj.*)

The authors are with the RCAM Laboratory, Djillali Liabes University, Sidi Bel Abbès 22000, Algeria (e-mails: mail@ycinhdj.com, yacine.hadjadj@univ-sba.dz; chakib@ipatz.dz; ne_taleb@yahoo.com, ne_taleb@univ-sba.dz; mazari_sarah@yahoo.fr, sarah.mazari@univ-sba.dz; elbahri82_m@yahoo.fr, mohamed.elbahri@univ-sba.dz; chikrelmezouar@gmail.com, chikrelmezouar@univ-sba.dz).

Digital Object Identifier 10.1109/TC.2023.3302731

more complex process that must meet at least the following four criteria: (1) speed: the dynamic allocator that allows allocation, reallocation, and memory release while consuming the fewest processor cycles is preferred; (2) memory consumption: the allocator that allocates with the least overhead is preferred; (3) fragmentation: at the heart of our study and contributions, the allocator that reduces memory dispersion is preferred; and (4) concurrency: the allocator that withstands concurrent processing without penalizing the application’s performance is preferred. Indeed, reducing memory fragmentation, which attempts to reduce the dispersion of allocated zones, is among the most important criteria for improving program behavior by reducing cache defects. In absolute terms, perfect contiguity of allocated memory zones achieves total elimination of fragmentation, which implies that all data structures are adjacent in memory. This configuration improves program performance as well as its readability. Furthermore, using such representations will allow developers to use GPUs that require data contiguity. However, adjacent structures do not support data resizing operations (memory reallocation and release), which involve moving and updating all references of data structures positioned after the structure affected by the resizing operation. Both operations (movement and updating) are considered time-consuming, and designers prefer to avoid them by proposing dispersed memory allocation strategies that do not involve moving data except for the memory zone affected by the resizing operation. Due to this difficulty, and as far as our knowledge allows us to affirm, only two contiguous memory allocators, CMA [1] and GCMA [2], are proposed in the state of the art, both exploited in the kernel, used for the allocation of physically contiguous memory, which is all static and therefore does not require any resizing operation. In this paper, we propose VCMalloc, a virtually contiguous custom memory allocator for the Microsoft Windows operating system, designed to allocate data structures confined in a virtually contiguous space that prioritizes memory fragmentation while reducing memory and processing overhead. To solve the two significant challenges posed by data resizing, we propose a strategy of virtual data movement by a remapping technique, avoiding memory copies that are considered time-consuming and energy intensive. For updating references, we offer a minimalist contiguous information system that accelerates search (by dichotomy). Considering that for particular data structures such as graphs, contiguity adds unnecessary complexity, we propose to avoid it by introducing a formalism of control over

contiguity. It should be noted that VCMalloc is not a drop-in replacement for Malloc, simply because the proposed formality requires a different memory interface signature and strategy for allocation, reallocation, and deallocation. Therefore, our main contributions are as follows:

- A novel approach for containers and hypercontainers to formalize virtual contiguity.
- A guaranteed virtual contiguity of data in the same hyper-container.
- A memory management system used for virtually contiguous memory allocation.
- New approaches for data reallocation using memory frames remapping, to avoid memory copy operations.

The rest of the paper is structured as follows: In Section II, we discuss the current state of the art in memory allocation and highlight the limitations of conventional memory allocators. In Section III, we present the general design of our memory allocator and discuss the strategies of allocation, reallocation, and deallocation used in the allocator. In Section IV, we discuss the important implementation details of VCMalloc and describe the new API created for VCMalloc and its usage. In Section V, we present the approaches we took to evaluate VCMalloc and provide interpretations of the results. Finally, in Section VI, we conclude the paper and discuss future directions for research in the field of memory allocation.

II. RELATED WORKS

The importance of virtual memory lies in its ability to improve memory utilization and performance in modern computer systems. By presenting a contiguous address space to applications, even when physical memory is not contiguous, virtually contiguous memory enables applications to make more efficient use of available memory. This is particularly critical in systems where memory is a scarce resource. Moreover, virtually contiguous memory allows applications to take advantage of hardware features like cache lines and prefetching, which can significantly enhance performance. As such, the widespread adoption of MMUs and TLBs has enabled the implementation of virtual memory in a wide range of devices, leading to improved memory utilization and performance in many computing scenarios, from embedded devices to server workloads [3], [4]. Even though virtual memory has offered solutions to various physical memory issues, the fragmentation problem has persisted for memory allocators.

A. Trends Towards Reducing Fragmentation

All the characteristics of a memory allocator (performance, memory usage, scalability) are either directly or indirectly troubled by the phenomena of fragmentation [5], or its consequences, like free lists, [6], [7], [8], [9], split and coalescing [10], [11], [12], [13], and compaction [14], which resulted from the development of many memory allocators over time [7], [9], [10], [12], [15], [16], [17], [18], [19]. In addition to the scope of allocators, many frameworks were provided to understand the behavior of an application in real-time with profiling [20], [21], [22], [23], or with

information that can be directly extracted from the source code (static analysis) [24], to design an optimal memory layout that reduces fragmentation. Modern memory allocators nowadays [25], separate allocations into different basis distributions. It can be thread based as in Hoard [15] and JEMalloc [17], size based as in SuperMalloc [18], CPU based as in TCMalloc [26], or page based as in MIMalloc [9]. These are clever techniques that significantly reduce memory fragmentation, improve cache locality, and optimize object allocation and deallocation, resulting in improved performance. However, these distributions do not necessarily provide the contiguity of objects in the same memory block (heap, arena, or chunk). Additionally, these distributions are completely abstracted by the allocator, with no control given to the user for their basis, leaving more room for improvements. Furthermore, general-purpose memory allocators are designed to allocate memory in a way that is convenient for the user, without necessarily considering the surrounding memory (location and layout of allocated objects). Therefore, when the surrounding memory is important and for the purpose of organizing allocations, users fall back to pre-allocating large contiguous memory blocks using general-purpose memory allocators [27]. Similarly, some memory allocators use several specialized memory allocation techniques in this same context, such as memory pooling [28], slab allocation [29], or buddy systems [6].

B. The Difficulties in Memory Preallocation

Pre-allocating memory is a commonly employed technique in both virtual and physical memory management. In fact, several studies have focused on enhancing physically contiguous memory allocation in Linux, with GCMalloc being a notable example [2], by introducing a logic of faster second-class clients. However, particularly from the virtual memory perspective, pre-allocating memory can present some challenges if not efficiently managed. In situations where the objects have dynamic memory requirements, the number of objects or their sizes can change frequently, making pre-allocation a hindrance rather than a help (Ex. Resizing an nD array) [30]. Additionally, pre-allocating a large container without considering the optimal size can lead to wasted space or the need to reallocate memory later, with the estimation being challenging in itself [31]. Re-allocating a large container can also be problematic for systems with limited memory resources, as it can lead to excessive memory usage and potential performance degradation (due to memory copy) [32]. Moreover, wasted memory resulting from pre-allocation cannot be reclaimed until the program terminates or all the allocated block is released, which can be problematic for long-running programs or applications that run on systems with limited memory resources. To overcome these drawbacks in pre-allocation, we provide an approach based on virtual contiguity. This approach must be consolidated by fast and lightweight reallocation strategies to get significantly reduced fragmentation.

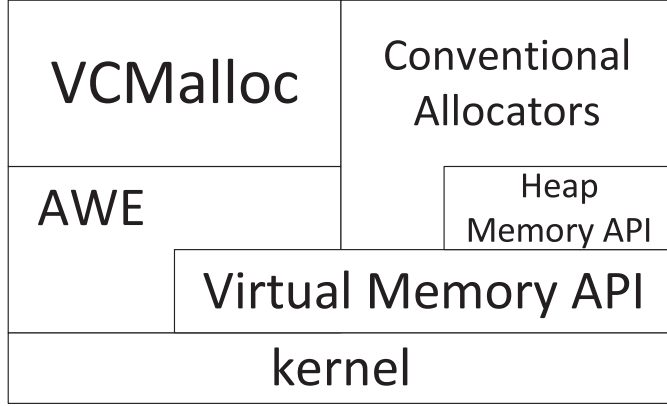


Fig. 1. The hierarchy of memory management in Windows, showing the relation between VCMalloc, virtual page management, and conventional memory allocators.

III. VCMALLOC DESIGN

The primary motivation behind the design of VCMalloc is on minimizing memory fragmentation and providing control over contiguity. The goal is to improve program performance and reduce memory usage. Usually, conventional allocators distribute allocations into blocks based on allocation size categories (small, medium, large), with a strategy that manages memory allocation and deallocation within each block. This approach leads to the grouping of contiguous memory zones based on the size of the allocated zones. Furthermore, traditional memory allocators on Windows often utilize the Heap API (HeapAlloc, HeapFree) or the Virtual Memory API (VirtualAlloc, VirtualFree) [33]. In contrast, VCMalloc leverages the AWE API (AllocateUserPhysicalPages, MapUserPhysicalPages) to manipulate the mappings between virtual and physical memory (Fig. 1). This approach offers increased flexibility in managing virtual and physical memory by leveraging the memory paging mechanisms of the underlying operating system [34].

Our approach focuses on empowering users with full control over the allocation and arrangement of memory zones, as well as ensuring virtual contiguity. To achieve this, we offer a straightforward formalism that allows users to apply the most appropriate strategy for their specific needs. This can include strategies such as perfect contiguity, distributing containers based on size or data type, or any other relevant approach.

A. The General Design of VCMalloc

On a conceptual level, the design of VCMalloc is guided by the following principles:

1) *Contiguity Control Formalism*: In our system, we conceptualize memory blocks as hypercontainers, each containing a finite set of memory zones referred to as containers. These hypercontainers are managed by a hypercontainer manager, which oversees multiple hypercontainers and their associated containers. We assume that all containers associated with the same hypercontainer are contiguous (Equation (1)), while the hypercontainers are dispersed. Containers residing in different hypercontainers are separated by the fact that hypercontainers

Algorithm 1 The strategy of reserving a new hypercontainer

```

1: function VC_HCALLOC( $s$ )
2:   Input
3:      $s$     Size of the requested HC
4:    $gran \leftarrow$  System allocation granularity
5:    $last\_addr \leftarrow$  Address of the last hypercontainer
6:    $last\_size \leftarrow$  Size of the last hypercontainer
7:    $addr \leftarrow last\_addr + last\_size$ 
8:   while  $addr$  is unavailable do
9:      $addr \leftarrow addr + gran$     ▷ Whenever the reservation at a certain address fails, try the next possible address.
10:  end while
11:  Create a new HC of address  $addr$  and size  $s$ 
12:  return  $addr$ 
13: end function

```

Algorithm 2 The strategy of allocating a new container

```

1: function VC_MALLOC( $hc, s$ )
2:   Input
3:      $hc$    The HC where the new container will reside
4:      $s$     Size of the new container
5:   if  $s >$  free physical space in  $c$  then
6:     Map new frames inside  $hc$ 
7:   end if
8:   Insert a container of size  $s$  inside  $hc$ 
9:   Return the new container's address
10: end function

```

are not always contiguous (Equation (2)). Through this simple formalism, users can produce completely contiguous data structures, replicate the allocation strategy adopted by any allocator, or define new ones.

$$C_1, C_2 \in HC \rightarrow C_1, C_2 \text{ Contiguous} \quad (1)$$

$$C_1 \in HC_1, C_2 \in HC_2 \rightarrow C_1, C_2 \text{ Dispersed} \quad (2)$$

Where:

C : is a container.

HC : is a hypercontainer.

2) *Virtual Space Reservation for Hypercontainers*: Before allocating containers, users must choose a hypercontainer or reserve a new one. This reservation is virtual and does not allocate any physical memory. This strategy is shown in Algorithm 1. The availability of a virtual memory address is determined by the operating system's capability to allocate and reserve a virtual memory at that specific location.

3) *Physical Pages Allocation for Containers*: Once a container is associated with a hypercontainer, enough physical memory will be inserted into the latter, if needed. (Algorithm 2) Upon this insertion, physical memory frames will be requested from the OS and then mapped to the container's virtual address space.

4) *Data Resizing by Memory Zone Mapping*: Contiguity of data requires a precise policy for reallocation that we propose

Algorithm 3 The strategy of reallocating a container in place

```

1: function VC_RESIZE(ptr, s)
2:   Input
3:     ptr   Pointer to the container to reallocate
4:     s     New size for the reallocation
5:   c ← The container corresponding to ptr
6:   hc ← The hypercontainer where c resides
7:   if c requires/releases pages then
8:     Copy guest bytes           ▷ Guest bytes are always
                                ≤ Sizepage − 1
9:     Map/Unmap pages inside hc
10:  end if
11:  Update hc ▷ Revise hc to shift containers positions to
                preserve their order after reallocation.
12: end function ▷ Since the reallocation happens in place,
                there is no need to return the same ptr

```

based on a strategy of remapping memory zones, replacing the conventional strategy of data copying. This deterministic approach results in fragmentation after reallocation, ranging from 0 bytes to a worst-case scenario of $S_P - 1$ bytes (as demonstrated in Equation (3)).

$$F = \begin{cases} S_P - S_{mod}, & \text{if } S_\Delta > 0 \cap S_{mod} \neq 0. \\ S_{mod}, & \text{otherwise.} \end{cases} \quad (3)$$

Where:

S_P : is the size of a memory page.

S_Δ : is the size difference $S_{new} - S_{old}$.

S_{mod} : is $|S_\Delta| \bmod S_P$.

We also define three reallocation strategies according to the nature of the data structures:

Reallocation in containers order: This strategy ensures the preservation of allocation order, thereby maintaining the integrity and linearity of data structures. The position of the reallocated container remains intact, prompting a subsequent updating process, where a rearrangement of containers allocated after the affected container is performed to maintain their original positions (Algorithm 3).

Reallocation out of containers order: In the context of non-linear data structures, the order of containers becomes less crucial. In this strategy, only the container directly affected by the reallocation will be moved to the end of the hypercontainer if necessary (Algorithm 4).

Reallocation of multiple containers: Resizing linear and regular data structures, such as multidimensional arrays, involves reallocating all dimensions. To accomplish this, we can create a container for each dimension within the structure. When resizing the multidimensional array, all containers can be reallocated simultaneously using a single remapping operation instead of individually reallocated M times, where M corresponds to the product of dimensions in the multidimensional array. The algorithm for this process is elaborated in Algorithm 5. During the initial traversal, we gather information about the number of pages released and requested. If we have an excess of

Algorithm 4 The strategy of reallocating a container and changing containers order if necessary

```

1: function VC_REALLOC(ptr, s)
2:   Input
3:     ptr   Pointer to the container to reallocate
4:     s     New size for the reallocation
5:   c ← The container pointed to by ptr
6:   hc ← The hypercontainer where c resides
7:   if can't reallocate c in place then ▷ This scenario
                                         typically occurs
                                         when a container
                                         needs to expand
                                         but lacks
                                         sufficient space
                                         in its current
                                         placement.
8:     Move c to the end of hc ▷ This includes moving
                                its pages and copying
                                its prior free bytes (refer to Fig. 2).
9:     Map/Unmap pages inside hc
10:  end if
11:  Update hc information system
12:  return ptr ▷ This process might change the container's
                address, we return ptr just as a normal
                realloc call does.
13: end function

```

pages, we can release them, whereas if we require more pages, we can request them from the operating system.

5) *Memory Deallocation:* To release memory, we propose three functions. The first function allows freeing the memory of a container, which we consider as a reallocation operation to 0 bytes. The second function is applied at the level of hypercontainers and frees the memory occupied by the hypercontainer as well as its information system (Algorithm 6). The last function completely destroys the memory occupied by all hypercontainers (Algorithm 7) and the entire information system.

6) *Page Size Selection:* Windows operating systems offer a default fixed memory page size of 4KB. However, larger size pages can be defined with a minimum size of 2MB. Using larger pages reduces the number of entries in the TLB tables, thus accelerating data access. However, these large pages must be reserved during the operating system's startup to avoid any risk of degrading system performance and increasing memory fragmentation [14]. As the goal of VCMalloc is to reduce fragmentation, we choose a minimum page size of 4KB, which avoids early memory reservation and maintains an acceptable level of performance for both VCMalloc and the operating system.

7) *VCMalloc Considerations:* After finalizing the overall design, we aim to outline the factors that every user should carefully consider in order to get the most out of VCMalloc.

Contiguous memory control: In contrast to conventional memory allocators that hide their internal mechanisms, VCMalloc grants users complete control through a straightforward

Algorithm 5 The strategy of resizing multiple containers

```

1: function VC_MRESIZE( $ptr, arr, n$ )
2:   Input
3:      $ptr$  The first container to reallocate
4:      $arr$  Array of sizes
5:      $n$  Number of containers to reallocate
6:      $c_{start} \leftarrow$  Container of pointer  $ptr$ 
7:      $c_{end} \leftarrow$  Container of pointer  $ptr + n - 1$ 
8:   for each container  $c_i$  from  $c_{start}$  to  $c_{end}$  do
9:     if  $c_i$  releases pages then
10:      Copy guest bytes  $\triangleright$  Guest bytes are always
11:                           $\leq Size_{page} - 1$ 
12:      Preserve pages released by  $c_i$ 
13:    end if
14:    if  $c_i$  requires pages then
15:      Gather the number of needed pages
16:    end if
17:    Update  $c_i$   $\triangleright$  Previous containers might have re-
18:                    leased pages,  $c_i$ 's position needs to
19:                    be shifted accordingly.
20:  end for
21:  Adjust preserved pages
22:  if containers require additional pages then
23:    for each container  $c_i$  from  $c_{start}$  to  $c_{end}$  do
24:      if  $c_i$  requires pages then
25:        Insert pages into  $hc$  for  $c_i$ 
26:        Copy guest bytes  $\triangleright$  Guest bytes are al-
27:                          ways  $\leq Size_{page} - 1$ 
28:      end if
29:      Update  $c_i$   $\triangleright$  Previous containers might
30:                    have allocated new pages,  $c_i$ 's
31:                    position needs to be shifted
32:                    accordingly.
33:    end for
34:  end if
35:  Update post-reallocation containers
36:  Update  $hc$ 
37: end function

```

Algorithm 6 The strategy of releasing a hypercontainer

```

1: function VC_HCFREE( $hc$ )
2:   Input
3:      $hc$  The HC to be freed
4:   Unmap and release frames associated with  $hc$ 
5:   Release the virtual space occupied by  $hc$ 
6:   Update the hypercontainer manager
7: end function

```

framework. When utilized correctly, this control can significantly enhance algorithms' performance. However, it necessitates a profound comprehension of memory cache behavior and introduces additional complexity in both design and implementation.

Algorithm 7 The strategy of releasing a hypercontainer manager

```

1: function VC_HCMFREE( $hcm$ )
2:   Input
3:      $hcm$  The hypercontainer manager to be freed
4:   for each hypercontainer  $hc_i$  in  $hcm$  do
5:     Unmap and release frames associated with  $hc_i$ 
6:     Release the virtual space occupied by  $hc_i$ 
7:   end for
8:   Release  $hcm$  data
9: end function

```

Concurrency: The current version does not incorporate any concurrent allocation strategies. Hence, if users intend to employ VCMalloc in multi-threaded applications, they must undertake the necessary precautions to avoid any problems related to concurrency.

B. VCMalloc Memory Management System

The VCMalloc memory management system consists of three main contexts: Container Context, Hyper Container Context, and Hyper Container Manager. The Container Context (Fig. 2), holds information about the size and location of an allocated memory block, including the user references pointing to that memory block, its index in the hypercontainer, and its address. It also calculates other parameters such as the offset, mapping address, next available address, and the number of pages. The Hyper Container Context (Fig. 3) manages multiple Container Context instances within a hypercontainer and contains information about the hypercontainer size, Container Context and PFN storage, and the number of allocated containers. It also has methods for performing operations such as returning the first and last container, inserting, resizing, or reallocating a container, and finding a container's index by its corresponding address. Finally, the Hyper Container Manager is located at the top of the VCMalloc hierarchy and is responsible for managing hypercontainers. It stores information about the Hyper Container Context instances and addresses of hypercontainers. Additionally, it has methods for finding a hypercontainer's index by its address and inserting a new hypercontainer. By utilizing this hierarchy, VCMalloc can offer virtual contiguity guarantees to memory allocated by the user, while still allowing for efficient management of the physical memory.

IV. VCMALLOC IMPLEMENTATION

For the implementation of VCMalloc contexts, three main data structures are created: VCMCC, VCMHCC, and VCMHCM. The VCMCC structure (Table I) is used to represent a single container. It contains information such as the container's index, pointer, size, and reference. The VCMHCC structure (Table II) is a higher-level container structure that manages multiple VCMCC structures. It keeps track of the number of containers and page frame numbers allocated and provides storage for container pointers and references. The

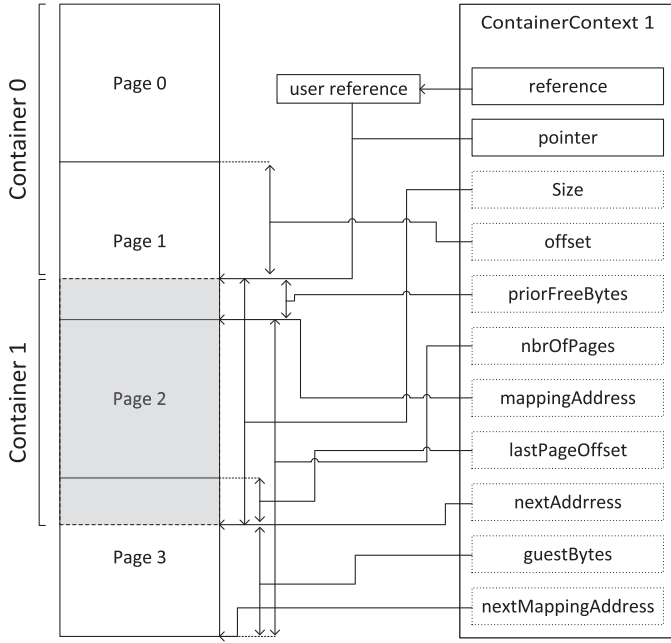


Fig. 2. The Container Context data structure.

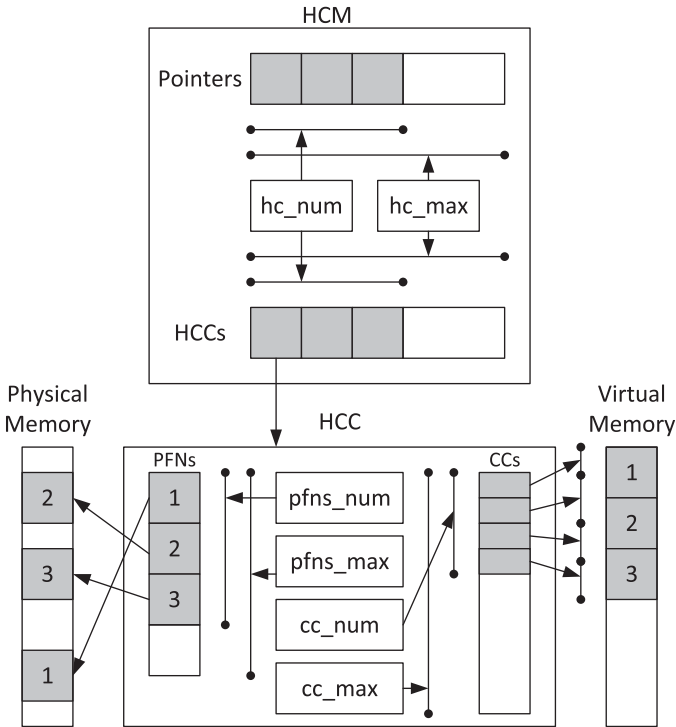


Fig. 3. The VCMalloc control plane (the organization of the HCM and the HCC data structures).

VCMHCM structure (Table III) is used to manage multiple VCMHCC structures, providing storage for higher-level container pointers and tracking the number of hypercontainers allocated. These data structures are essential to the efficient operation of the VCMalloc allocator. By providing a way to manage memory at multiple levels of abstraction, they allow the allocator to adapt to the needs of different memory management operations.

TABLE I
VCMCC

Field	Description
index	index of the container
pointer	pointer to the container
refs	reference to the container
size	size of the container

TABLE II
VCMHCC

Field	Description
cc_num	number of containers allocated
cc_max	maximum number of containers
pfn_num	number of page frame numbers allocated
pfn_max	maximum number of page frame numbers
pointer_storage	storage for container pointers
ref_storage	storage for container references
last_c_size	size of the last container allocated
pfn_storage	storage for page frame numbers

TABLE III
VCMHCM

Field	Description
hc_num	number of HCC allocated
hc_max	maximum number of HCC
hcc_storage	storage for HCC
p_storage	storage for pointers of hypercontainers
last_hc_size	size of the last HCC allocated

TABLE IV
VCMALLOC APIs

API	Description
Allocation	
vc_halloc(s)	Pre-allocates a virtual hypercontainer of size s
vc_malloc(hc,s)	Allocates s bytes in the hypercontainer hc
vcr_malloc(hc,ref,s)	Allocates s bytes in the hypercontainer hc and tracks the reference ref for this allocation
vca_malloc(s)	Allocates s bytes
vcar_malloc(ref,s)	Allocates s bytes and tracks the reference ref for this allocation
Reallocation	
vc_resize(ptr,s)	Resizes container ptr in place, with the new size s
vc_mresize(ptr,sizes,s)	Resizes s consecutive containers with the new size sizes[i], in place, starting with container ptr
vc_realloc(ptr,s)	Resizes container ptr with the new size s, if it can't be resized in place, move the container to the end of the hypercontainer
Deallocation	
vc_hcfree(hc)	Deallocates the entire hypercontainer hc
vc_hcmfree(hcm)	Frees all the hypercontainers in the Hyper Container Manager hcm

For the implementation of the VCMalloc APIs (see Table IV), we used the AWE APIs from MS Windows. Requests for virtual address ranges are performed using the MS Windows's API "VirtualAlloc" [35]. Then, VCMalloc maps the non-contiguous physical frames to the reserved virtual address range using the "MapUserPhysicalPages" [36] API. The physical frames are exchanged with the operating system using the "AllocateUserPhysicalPages" [37] and "FreeUserPhysicalPages" [38] API calls. The storage inside VCMalloc's memory management system is allocated using the "HeapAlloc" API, and it grows implicitly, using the "HeapRealloc" API call. This is an expensive operation.

Therefore, VCMalloc offers an API to pre-extend or create contexts of containers and hypercontainers with enough storage to prevent multiple implicit reallocations. VCMalloc's source code is available as a library of about 1500 lines of code, it can be compiled as an object file with programs or linked as a library. It is available in C and C++. at: <https://github.com/ycinhdj/vcmalloc>.

V. EVALUATION

The current version of VCMalloc does not implement any concurrent allocation strategy. Therefore, for our evaluation, we focus on the remaining three criteria; speed, fragmentation, and memory consumption. We propose two sets of evaluations, one for basic functions and one for real-world applications. We compare VCMalloc results to those of the default Windows 11 system allocator and to those of the most recent MIMalloc allocator, which has demonstrated greater performance compared to other allocators. We obtained our results on a workstation equipped with Windows 11 operating system and MSVC++ compiler. Memory consumption statistics were obtained using MS Windows's PSAPI, while cache misses metrics were obtained using the Intel PCM library. Finally, since the SPEC-CPU 2017 benchmark does not allow changes to the code, we extracted the code from selected applications, modified it to support VCMalloc, and recompiled it for all three allocators using the MSVC++ compiler.

A. Basic Operations Evaluation Series

This series is intended for evaluating memory allocation, re-allocation, and release operations in the context of regular data structures. Two-dimensional matrices, widely used in computer vision [39], image processing [40] and machine learning [41], were chosen as the target structure for evaluation. There are two representations of 2D matrices: the first one using multidimensional pointers or indexed arrays (Listing 1), and the second one using affine arrays (a one-dimensional vector that emulates the behavior of a two-dimensional matrix through a data access technique) (Listing 2). The first representation was chosen, as it is the only one that supports memory resizing operations (Listing 3). For a square matrix with dimensions $[M][M]$, the multidimensional pointer representation requires $M + 1$ allocations. In this evaluation, we vary M from 100 to 10000, with a step of 100. We then double the size of each structure using a reallocation operation to reach a size of $[M][2M]$. In the end, we release the structures. During each allocation and reallocation operation, we measure the minimum, maximum, and average fragmentation between successive allocations or reallocations, the peak memory usage, and the number of CPU cycles for all three allocators (VCMalloc, MIMalloc, and Malloc). This evaluation series aims to provide insights into the performance and behavior of memory allocation and deallocation operations for two-dimensional matrices using multidimensional pointers.

Memory Allocation: An analysis of the obtained data shows a consistent trend based on the size of the structure. To illustrate this, we present a sample of the results obtained in Table V, which includes a representative case of small-sized $[100][100]$,

Listing 1. 2D array allocation and access using indexed array technique

```
double **array = (double**)malloc(M * sizeof(double*));
for(int i = 0; i < M; i++)
    array[i] = (double*)malloc(M * sizeof(double));
// Access array elements using indexing
for(int i = 0; i < M; i++)
    for(int j = 0; j < M; j++)
        array[i][j] = i + j;
```

Listing 2. 2D array allocation and access using affine array technique

```
double *array = (double*)malloc(M * M * sizeof(double));
// Access array elements using affine formula
for(int i = 0; i < M; i++)
    for(int j = 0; j < M; j++)
        array[i * M + j] = i + j;
```

Listing 3. Resizing a 2D array using indexed array technique

```
// Resize the array to Mx2M
for(int i = 0; i < M; i++)
    array[i] = (double*)realloc(array[i], 2 * M * sizeof(double));
```

medium-sized $[1000][1000]$, and large-sized $[10,000] \times [10,000]$ structures. The results confirm previous research that indicates the superiority of MIMalloc over Malloc, with MIMalloc demonstrating greater performance (clk: clock cycles) over Malloc by 2 to 18% between small, medium-sized, and large-sized allocations. However, this comes at the cost of 12 to 225% higher memory consumption (cr_mem: Current Memory). We also observe that MIMalloc's allocation strategy, which distributes zones based on their sizes, offers better fragmentation than Malloc. However, only VCMalloc produces a perfectly contiguous allocation without any fragmentation (frg_avg: Average Fragmentation), which can significantly improve the data structure's behavior. It shows the best results in terms of processor cycles (up to 28% improvement for large-sized allocations compared to MIMalloc) and memory consumption that is very similar to that observed for Malloc.

Memory Reallocation: Many applications, such as image processing, computer vision, and multi-linear algebra, require resizing allocated matrices or multiple reallocations [42], [43], [44]. In the case of VCMalloc, the second strategy of ordered multiple reallocations proposed by VCMalloc is chosen, as shown in Table VI which also presents a sample of representative results obtained from the reallocation of small $[100][100]$, medium $[1000][1000]$, and large $[10000][10000]$ structures, and which will respectively lead to sizes of $[200][100]$, $[2000][1000]$, and $[20000][10000]$. The trend is the same, where MIMalloc performs better at the cost of consuming much more memory. On the other hand, the multiple reallocation strategy proposed by VCMalloc (Listing 4) shows better performance compared to the MIMalloc strategy, while displaying memory consumption very close to Malloc during allocation operations.

Memory Deallocation: In general, memory deallocation times are shorter than memory allocation and reallocation times, as described in Table VII. The f_cr_mem column shows the memory usage after deallocation, and the r_cr_mem column shows the memory usage before deallocation. The f_clk column

TABLE V
SAMPLE RESULTS OF DATA ALLOCATION PERFORMANCE BASED ON STRUCTURE SIZE

Allocator	Size	clk	cr_mem	pk_mem	frg_min	frg_max	frg_avg	L3_cch
VCMalloc		1.20B	774.56Mi	774.56Mi	0.00	0.00	0.00	120.44K
Malloc	10Kx10K	1.69B	777.83Mi	777.83Mi	16.00	1.62Gi	1.71Mi	292.72K
MIMalloc		1.66B	876.02Mi	876.02Mi	1.88Ki	545.88Ki	8.54Ki	312.89K
VCMalloc	1Kx1K	16.59M	17.70Mi	17.70Mi	0.00	0.00	0.00	2.21K
Malloc		20.80M	18.29Mi	18.29Mi	16.00	36.79Mi	71.54Ki	10.87K
MIMalloc		17.03M	42.34Mi	42.34Mi	192.00	192.00	192.00	2.99K
VCMalloc	100x100	4.58M	10.02Mi	10.02Mi	0.00	0.00	0.00	291.00
Malloc		5.54M	10.11Mi	10.11Mi	16.00	18.42Ki	44.69Ki	868.00
MIMalloc		5.05M	32.90Mi	32.90Mi	96.00	1.09Ki	106.00	827.00

TABLE VI
COMPARISON OF DATA REALLOCATION STRATEGIES FOR VCMALLOC, MIMALLOC, AND MALLOC

Allocator	Old size	New size	r_clk	r_cr_mem	r_pk_mem	r_frg_min	r_frg_max	r_frg_avg	r_L3_cch_ms
VCMalloc	10Kx10K	20Kx10K	2.53B	1.52Gi	1.53Gi	1.88Ki	1.88Ki	1.88Ki	856.83K
Malloc			3.03B	1.51Gi	1.51Gi	16.00	893.09Mi	792.52Mi	4.01M
MIMalloc			2.86B	1.86Gi	1.86Gi	35.75Ki	1.01Gi	341.12Mi	394.17K
VCMalloc	1Kx1K	2Kx1K	23.25M	25.58Mi	25.58Mi	192.00	192.00	192.00	73.13K
Malloc			31.30M	26.70Mi	26.70Mi	400.00	45.44Mi	26.29Mi	36.19K
MIMalloc			30.09M	42.34Mi	42.34Mi	384.00	8.06Mi	4.40Mi	39.95K
VCMalloc	100x100	200x100	2.98M	10.41Mi	10.41Mi	3.22Ki	3.22Ki	3.22Ki	921.00
Malloc			3.10M	10.25Mi	10.25Mi	16.00	422.04Mi	231.54Mi	411.00
MIMalloc			3.04M	42.43Mi	42.43Mi	192.00	1.19Ki	212.00	334.00

Listing 4. Resizing a 2D array using the multiple reallocation strategy proposed by VCMalloc

```
// Resize the array to Mx2M
size_t *sizes = (size_t*)malloc(sizeof(size_t)*M);
for(int i = 0; i < M; i++)
    sizes[i] = 2*M*sizeof(double);
vca_mresize(array, sizes, M);
```

indicates the number of CPU cycles taken for the deallocation process. The `f_l3_cch_ms` column represents the number of cache misses in the L3 cache. We can see that VCMalloc has the highest `f_prtg` (percentage of freed memory) for all matrix sizes. On average, it frees around 99.36% of the allocated memory, compared to 99.22% for Malloc and only 0.30% for MIMalloc, indicating that it completely frees the virtual space and returns the page frames to the OS. Malloc shows a reasonable deallocation performance with a moderate cache miss of L3, and moderate CPU cycles taken for deallocation. However, the `f_prtg` column shows that Malloc has a slightly lower percentage of memory being returned after deallocation compared to VCMalloc. On the other hand, VCMalloc frees both the allocated memory and its information system, resulting in degraded performance for deallocation compared to Malloc. The `f_prtg` column also shows that MIMalloc has a very low percentage of memory being returned after deallocation, ranging from 0% to 0.3%. Therefore, it has the lowest `f_clk` (CPU cycles for deallocation) and the lowest cache miss rate (`f_L3_cch_ms`) for all matrix sizes.

B. Real-World Application Evaluation Series

For real-world applications, two major categories are distinguished: Applications written with an awareness of contiguity

Listing 5. Allocating 2D matrix with VCMalloc for the matrix multiplication evaluation

```
char* hc = vc_hcalloc(hc_size);
//SPF: Single hc, Pointers First
double **A = vc_malloc(hc, sizeof(double)*M);
double **B = vc_malloc(hc, sizeof(double)*M);
double **C = vc_malloc(hc, sizeof(double)*M);
for(size_t i; i < M; i++)
    A[i] = (double*)vc_malloc(hc, sizeof(double)*M);
for(size_t i; i < M; i++)
    B[i] = (double*)vc_malloc(hc, sizeof(double)*M);
for(size_t i; i < M; i++)
    C[i] = (double*)vc_malloc(hc, sizeof(double)*M);
```

control, and other real-world applications that use the default allocation strategy.

Applications with awareness of contiguity control: Among the applications used to evaluate the behavior of data structure cache memories, we propose to cite the matrix multiplication [45]. For each allocator, we propose to create three data structures *A*, *B*, and *C* representing two-dimensional square matrices $[M][M]$ and to calculate the product $C = A * B$ using the same naive algorithm. For data structures allocated by VCMalloc, we choose the optimal structure with a single hypercontainer, containing pointers and data, with all the pointers allocated first (Listing 5). For this evaluation, we vary the size of matrices $M \times M$ in the interval $[4000 - 8000]$ by a step of 1000. VCMalloc, with its contiguous distribution, allows for two memory access models: multidimensional pointer, also referred to as indexed array (Listing 6), or affine array, a technique that emulates the behavior of 2D matrices using a 1D array (Listing 7). As for other allocators (MIMalloc and Malloc), the produced data structures are scattered and we cannot make any assumptions about their memory allocation (Only an indexed array access pattern is possible).

TABLE VII
COMPARISON OF MEMORY DEALLOCATION WITH MALLOC, VCMALLOC, AND MIMALLOC

Allocator	Size	r_cr_mem	f_cr_mem	f_clk	f_L3_cch_ms	f_pretg
VCMalloc	10Kx10K	1.52Gi	9.98Mi	1.56B	451.24K	99.36
Malloc		1.51Gi	12.04Mi	967.66M	395.69K	99.22
MIMalloc		1.86Gi	1.85Gi	12.64M	2.69K	0.30
vcmalloc	1Kx1K	25.58Mi	10.04Mi	45.74M	43.36K	60.77
malloc		26.70Mi	10.49Mi	9.22M	7.79K	60.71
mimalloc		42.34Mi	42.34Mi	6.98M	3.21K	0.00
vcmalloc	100x100	10.41Mi	9.95Mi	3.07M	362.00	4.50
malloc		10.25Mi	10.20Mi	2.75M	138.00	0.53
mimalloc		42.43Mi	42.43Mi	2.70M	98.00	0.00

Listing 6. Indexed array access pattern of matrices allocated with VCMalloc

```

for (size_t i = 0; i < M; i++)
    for (size_t j = 0; j < M; j++){
        C[i][j] = 0;
        for (size_t k = 0; k < M; k++){
            C[i][j] += A[i][k] * B[k][j];
        }
    }

```

Listing 7. Affine array access pattern for matrices allocated with VCMalloc

```

for (size_t i = 0; i < M; i++)
    for (size_t j = 0; j < M; j++){
        C[i*M + j] = 0;
        for (size_t k = 0; k < M; k++){
            C[i*M + j] += A[i*M + k] * B[k*M + j];
        }
    }

```

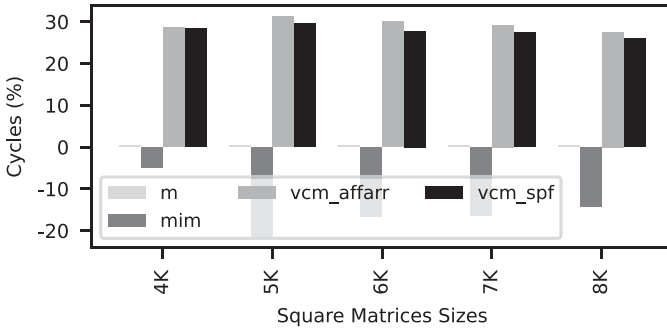
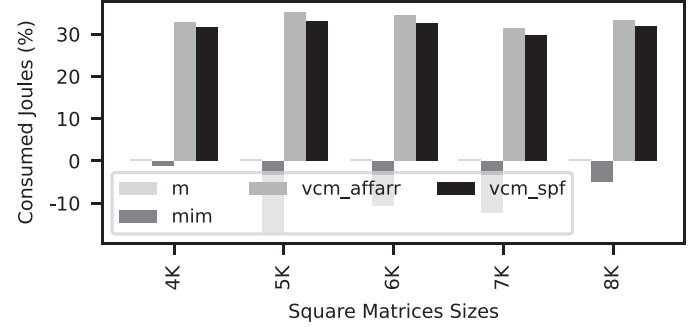


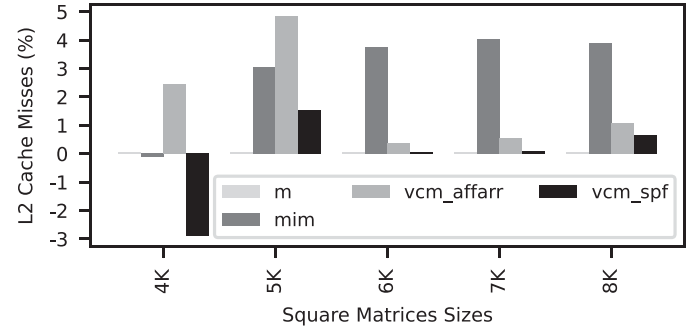
Fig. 4. Memory allocation impact on matrix multiplication performance (CPU cycles): comparing Malloc, MIMalloc, and VCMalloc.

Figs. 4, 5, 6, and 7 respectively represent the number of clock cycles, energy consumption, L2 cache misses, and L3 cache misses as a function of matrix size. Due to the significant differences observed between different matrix sizes, we display the relative optimization rate compared to Malloc (the default allocator) calculated by the following formula: $(AllocatorMeasurement - MallocMeasurement / MallocMeasurement) * 100$. A positive percentage indicates an improvement rate, while a negative percentage indicates a degradation rate. The study found that VCMalloc outperformed Malloc and MIMalloc in terms of cycle efficiency, as illustrated in Fig. 4. VCMalloc's superiority



	min	mean	max
m	0%	0%	0%
mim	-17%	-9%	-1%
vcm_affarr	31%	33%	35%
vcm_spf	30%	32%	33%

Fig. 5. Memory allocation impact on matrix multiplication performance (energy consumption): comparing Malloc, MIMalloc, and VCMalloc.



	min	mean	max
m	0%	0%	0%
mim	0%	3%	4%
vcm_affarr	0%	2%	5%
vcm_spf	-3%	0%	2%

Fig. 6. Memory allocation impact on matrix multiplication performance (L2 cache miss): comparing Malloc, MIMalloc, and VCMalloc.

is attributed to the contiguous memory allocation it provides. The study also demonstrated that VCMalloc exhibits better cache misses behavior (Figs. 6 and 7) and lower energy consumption (Fig. 5).

Applications with no awareness of contiguity control: This series is intended for applications that have no awareness of contiguity control, for which we limit ourselves to creating

TABLE VIII
PERFORMANCE COMPARISON OF DIFFERENT ALLOCATORS FOR SELECTED SPECCPU APPLICATIONS

Application	Allocator	Cycles	L2Misses	L3Misses	L2HitRatio	L3HitRatio	Memory
ImageValidator	VCMalloc	2.29K	30.10M	2.00M	0.70	0.93	2.02Gi
	Malloc	2.35K	34.34M	2.50M	0.56	0.93	2.02Gi
	MIMalloc	2.33K	22.70M	2.39M	0.78	0.89	2.10Gi
520.omnetpp_r	VCMalloc	440.77K	11.57B	4.14B	0.53	0.64	259.12Mi
	Malloc	446.08K	11.70B	4.16B	0.56	0.64	260.73Mi
	MIMalloc	442.39K	11.54B	4.13B	0.57	0.64	312.32Mi
ldecod_r	VCMalloc	15.02K	151.96M	48.31M	0.70	0.67	142.83Mi
	Malloc	15.41K	157.24M	48.63M	0.70	0.68	142.85Mi
	MIMalloc	15.91K	161.27M	47.29M	0.72	0.69	166.50Mi

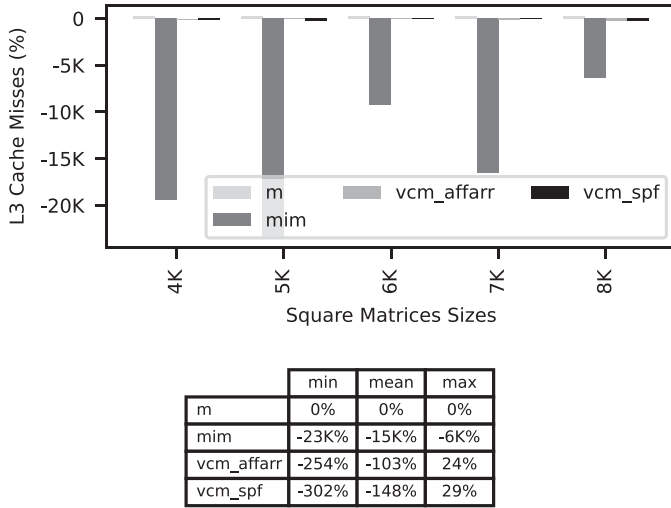


Fig. 7. Memory allocation impact on matrix multiplication performance (L3 cache miss): comparing Malloc, MIMalloc, and VCMalloc.

a single hypercontainer that will group all required memory zones. For reallocation and as we have no constraints regarding the order of allocations, we opt for the strategy of an out-of-order reallocation. In this sub-series, we propose to evaluate the performance of VCMalloc (memory consumption and CPU resources) compared to the two other allocators (MIMalloc and Malloc) in SPECCPU 2017 applications written without any zone contiguity strategy. Since the signature of VCMalloc is different from the other allocators due to the introduction of the hypercontainer concept, using VCMalloc requires syntax adaptations of real-world applications' source code, which can be complex and time-consuming given the code's complexity. Therefore, we adapted the following applications, all present in the SPEC CPU 2017 benchmark: the Omnet++ benchmark, and two utilities used by the test suite (imagevalidate, and ldecod).

As shown in Table VIII, although these applications have no awareness of VCMalloc, the results obtained demonstrate slightly better performance for MIMalloc over Malloc (0.8%), as previously reported in the literature. However, this comes at the cost of higher memory consumption (18% additional memory usage for ldecod and 20% for Omnetpp++). VCMalloc offers the best compromise, delivering better performance than MIMalloc without any excessive memory consumption.

VI. CONCLUSION

This research paper presents VCMalloc, a custom memory allocator for Microsoft Windows that enables the creation of

virtually contiguous data structures, reducing physical memory waste and the need for data copying during reallocation. Our experimental results, which include real-world applications such as naive matrix multiplication and SPECCPU 2017 benchmarks, as well as micro-benchmarks consisting of regular data structures, validate the effectiveness of our proposed memory allocation, reallocation, and deallocation strategies. The success of VCMalloc has inspired us to expand its concepts to other operating systems and seek further improvements by integrating an underlying physically contiguous memory allocator such as GCMA [2]. Additionally, we plan to study and design compatible irregular data structures that take into account VCMalloc's capabilities and explore how VCMalloc techniques can enhance GPGPU applications.

REFERENCES

- [1] M. Nazarewicz, "Contiguous memory allocator," in *Proc. LinuxCon Europe*, 2012.
- [2] S. Park, M. Kim, and H. Y. Yeom, "GCMA: Guaranteed contiguous memory allocator," *IEEE Trans. Comput.*, vol. 68, no. 3, pp. 390–401, Mar. 2019.
- [3] S. Mittal, "A survey of techniques for architecting TLBs," *Concurrency Comput., Pract. Exp.*, vol. 29, no. 10, 2017, Art. no. e4061. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1002/cpe.4061>
- [4] J.-H. Choi, J.-H. Lee, S.-W. Jeong, S.-D. Kim, and C. Weems, "A low power TLB structure for embedded systems," *IEEE Comput. Archit. Lett.*, vol. 1, no. 1, p. 3, Feb. 2002.
- [5] P. Wilson, M. Johnstone, M. Neely, and D. Boles, "Dynamic storage allocation: A survey and critical review," in *Proc. Int. Workshop Memory Manage.*, Oct. 1995, vol. 986, pp. 1–116.
- [6] M. S. Johnstone and P. R. Wilson, "The memory fragmentation problem: Solved?" *SIGPLAN Notices*, vol. 34, no. 3, pp. 26–36, Oct. 1998, doi: <https://doi.org/10.1145/301589.286864>.
- [7] S. Schneider, C. D. Antonopoulos, and D. S. Nikolopoulos, "Scalable locality-conscious multithreaded memory allocation," in *Proc. 5th Int. Symp. Memory Manage.*, New York, NY, USA: ACM, Jun. 2006, pp. 84–94, doi: <https://doi.org/10.1145/1133956.1133968>.
- [8] R. Shahriyar, S. M. Blackburn, X. Yang, and K. S. McKinley, "Taking off the gloves with reference counting Immix," in *Proc. ACM SIGPLAN Int. Conf. Object Oriented Program. Syst. Lang. Appl.*, New York, NY, USA: ACM, Oct. 2013, pp. 93–110, doi: <https://doi.org/10.1145/2509136.2509527>.
- [9] D. Leijen, B. Zorn, and L. de Moura, "Mimalloc: Free list sharding in action," in *Proc. Program. Lang. Syst.*, A. W. Lin, Ed., Cham, Switzerland: Springer International Publishing, 2019, pp. 244–265.
- [10] L. Doug, "A memory allocator," Accessed: Dec. 28, 2021. [Online]. Available: <http://gee.cs.oswego.edu/dl/html/malloc.html>
- [11] M. S. Johnstone and P. R. Wilson, "The memory fragmentation problem: Solved?" *SIGPLAN Notices*, vol. 34, no. 3, pp. 26–36, Oct. 1998, doi: <https://doi.org/10.1145/301589.286864>.
- [12] Y. Hasan and M. Chang, "A study of best-fit memory allocators," *Comput. Lang., Syst. Struct.*, vol. 31, no. 1, pp. 35–48, Apr. 2005. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1477842404000211>

- [13] R. McIlroy, P. Dickman, and J. Sventek, "Efficient dynamic heap allocation of scratch-pad memory," in *Proc. Int. Symp. Memory Manage.*, New York, NY, USA: ACM, Jun. 2008, pp. 31–40, doi: <https://doi.org/10.1145/1375634.1375640>.
- [14] A. Panwar, A. Prasad, and K. Gopinath, "Making huge pages actually useful," in *Proc. 23rd Int. Conf. Architectural Support Program. Lang. Oper. Syst.*, New York, NY, USA: ACM, Mar. 2018, pp. 679–692, doi: <https://doi.org/10.1145/3173162.3173203>.
- [15] E. D. Berger, K. S. McKinley, R. D. Blumofe, and P. R. Wilson, "Hoard: A scalable memory allocator for multithreaded applications," *SIGARCH Comput. Archit. News*, vol. 28, no. 5, pp. 117–128, Nov. 2000, doi: <https://doi.org/10.1145/378995.379232>.
- [16] Y. Feng and E. D. Berger, "A locality-improving dynamic memory allocator," in *Proc. Workshop Memory Syst. Perform.*, New York, NY, USA: ACM, Jun. 2005, pp. 68–77, doi: <https://doi.org/10.1145/1111583.1111594>.
- [17] J. Evans, "A scalable concurrent malloc(3) implementation for FreeBSD," 2006, <https://papers.freebsd.org/2006/bsdcan/evans-jemalloc.files/evans-jemalloc-paper.pdf>.
- [18] B. C. Kuzmaul, "SuperMalloc: A super fast multithreaded malloc for 64-bit machines," *SIGPLAN Notices*, vol. 50, no. 11, pp. 41–55, Jun. 2015, doi: <https://doi.org/10.1145/2887746.2754178>.
- [19] A. H. Hunter, C. Kennelly, D. Gove, P. Ranganathan, P. J. Turner, and T. J. Moseley, "Beyond malloc efficiency to fleet efficiency: A hugepage-aware memory allocator," in *Proc. 15th USENIX Symp. Oper. Syst. Des. Implementation (OSDI)*, 2021, pp. 257–273.
- [20] B. Calder, C. Krintz, S. John, and T. Austin, "Cache-conscious data placement," *SIGOPS Oper. Syst. Rev.*, vol. 32, no. 5, pp. 139–149, Oct. 1998, doi: <https://doi.org/10.1145/384265.291036>.
- [21] M. L. Seidl and B. G. Zorn, "Segregating heap objects by reference behavior and lifetime," *SIGOPS Oper. Syst. Rev.*, vol. 32, no. 5, pp. 12–23, Oct. 1998, doi: <https://doi.org/10.1145/384265.291012>.
- [22] J. Savage and T. M. Jones, "HALO: Post-link heap-layout optimisation," in *Proc. 18th ACM/IEEE Int. Symp. Code Gener. Optim.*, New York, NY, USA: ACM, Feb. 2020, pp. 94–106, doi: <https://doi.org/10.1145/3368826.3377914>.
- [23] D.-J. Oh et al., "MaPHeA: A lightweight memory hierarchy-aware profile-guided heap allocation framework," in *Proc. 22nd ACM SIGPLAN/SIGBED Int. Conf. Lang., Compilers, Tools Embedded Syst.*, New York, NY, USA: ACM, Jun. 2021, pp. 24–36, doi: <https://doi.org/10.1145/3461648.3463844>.
- [24] C. Latner and V. Adve, "Automatic pool allocation: Improving performance by controlling data structure layout in the heap," *SIGPLAN Notices*, vol. 40, no. 6, pp. 129–142, Jun. 2005, doi: <https://doi.org/10.1145/1064978.1065027>.
- [25] X. Ouyang and Y. Zhu, "wfsan: Wait-free dynamic memory management," *ACM Trans. Embedded Comput. Syst.*, vol. 21, no. 4, pp. 43:1–43:26, Aug. 2022, doi: <https://doi.org/10.1145/3533724>.
- [26] "TCMalloc: Thread-caching malloc." TCMalloc. Accessed: Jan. 22, 2022. [Online]. Available: <https://google.github.io/tcmalloc/overview.html>
- [27] G. Pleiss, D. Chen, G. Huang, T. Li, L. van der Maaten, and K. Q. Weinberger, "Memory-efficient implementation of DenseNets," Jul. 2017. [Online]. Available: <http://arxiv.org/abs/1707.06990>
- [28] Q. Zhao, R. Rabbah, and W.-F. Wong, "Dynamic memory optimization using pool allocation and prefetching," *SIGARCH Comput. Archit. News*, vol. 33, no. 5, pp. 27–32, Dec. 2005, doi: <https://doi.org/10.1145/1127577.1127584>.
- [29] P. Liétar et al., "snmalloc: A message passing allocator," in *Proc. ACM SIGPLAN Int. Symp. Memory Manage.*, New York, NY, USA: ACM, Jun. 2019, pp. 122–135, doi: <https://doi.org/10.1145/3315573.3329980>.
- [30] R. Patgiri, S. K. Borgohain, and S. Pal, "Elastica a large scale elastic array data structure," in *Proc. Int. Conf. Comput. Commun. Inf. (ICCCI)*, Jan. 2018, pp. 1–7.
- [31] W. Zhao, Z. Wang, and Y. Luo, "Dynamic memory balancing for virtual machines," *SIGOPS Oper. Syst. Rev.*, vol. 43, no. 3, pp. 37–47, Jul. 2009, doi: <https://doi.org/10.1145/1618525.1618530>.
- [32] M. Maule, J. L. D. Comba, R. Torchelsen, and R. Bastos, "Memory-efficient order-independent transparency with dynamic fragment buffer," in *Proc. 25th SIBGRAPI Conf. Graph., Patterns Images*, Aug. 2012, pp. 134–141.
- [33] B. Catlin, J. E. Hanrahan, M. E. Russinovich, D. A. Solomon, and A. Ionescu, *System Architecture, Processes, Threads, Memory Management, and More*, 7th ed., Redmond, WA, USA: Microsoft, 2017.
- [34] N. Douglas, "User mode memory page allocation: A silver bullet for memory allocation?" *Comput. Res. Repository*, May 2011, [arxiv:1105.1811](https://arxiv.org/abs/1105.1811).
- [35] "VirtualAlloc function (memoryapi.h) - Win32 apps." Microsoft. Accessed: Feb. 3, 2022. [Online]. Available: <https://docs.microsoft.com/en-us/windows/win32/api/memoryapi/nf-memoryapi-virtualalloc>
- [36] "MapUserPhysicalPages function (memoryapi.h) - Win32 apps." Microsoft. Accessed: Feb. 3, 2022. [Online]. Available: <https://docs.microsoft.com/en-us/windows/win32/api/memoryapi/nf-memoryapi-mapuserphysicalpages>
- [37] "AllocateUserPhysicalPages function (memoryapi.h) - Win32 apps." Microsoft. Accessed: Feb. 3, 2022. [Online]. Available: <https://docs.microsoft.com/en-us/windows/win32/api/memoryapi/nf-memoryapi-allocateuserphysicalpages>
- [38] "FreeUserPhysicalPages function (memoryapi.h) - Win32 apps." Microsoft. Accessed: Feb. 3, 2022. [Online]. Available: <https://docs.microsoft.com/en-us/windows/win32/api/memoryapi/nf-memoryapi-freeuserphysicalpages>
- [39] S. Yoon, A. Chandra, and G. Vahedi, "Stripenn detects architectural stripes from chromatin conformation data using computer vision," *Nature Commun.*, vol. 13, no. 1, Mar. 2022, Art. no. 1602. [Online]. Available: <https://www.nature.com/articles/s41467-022-29258-9>
- [40] A. Daoui, H. Karmouni, M. Sayyouri, and H. Qjjada, "Stable analysis of large-size signals and images by Racah's discrete orthogonal moments," *J. Comput. Appl. Math.*, vol. 403, Mar. 2022, Art. no. 113830. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0377042721004520>
- [41] J. B. Harley and D. Sparkman, "Machine learning and NDE: Past, present, and future," *AIP Conf. Proc.*, vol. 2102, no. 1, May 2019, Art. no. 090001. [Online]. Available: <https://aip.scitation.org/doi/abs/10.1063/1.5099819>
- [42] B. P. Das, M. Biswal, A. Panigrahi, and M. Okade, "CNN based image resizing detection and resize factor classification for forensic applications," in *Proc. 2nd Int. Conf. Range Technol. (ICORT)*, Aug. 2021, pp. 1–6.
- [43] K. ur Rehman, J. Li, Y. Pei, A. Yasin, S. Ali, and T. Mahmood, "Computer vision-based microcalcification detection in digital mammograms using fully connected depthwise separable convolutional neural network," *Sensors*, vol. 21, no. 14, Jan. 2021, Art. no. 4854. [Online]. Available: <https://www.mdpi.com/1424-8220/21/14/4854>
- [44] J. Rumengan, T. Y. Zhuo, and C. Sanderson, "PyArmado: A streamlined linear algebra library for Python," *J. Open Source Softw.*, vol. 6, no. 66, Oct. 2021, Art. no. 3051. [Online]. Available: <https://joss.theoj.org/papers/10.21105/joss.03051>
- [45] A.-H. Badawy, A. Aggarwal, D. Yeung, and C.-W. Tseng, "The efficacy of software prefetching and locality optimizations on future memory systems," *J. Instruction-Level Parallelism*, vol. 6, pp. 1–36, Jul. 2004.



Things. He was awarded the valedictorian title upon the M.S. graduation from Djillali Liabes University.



and distributed computing, operating systems, and networking. He is also an International Expert and Solutions Architect in networking, security, and data centers.

Yacine Hadjadj received the Master of Science degree in embedded systems from Djillali Liabes University, Sidi Bel Abbes, Algeria. He is currently working toward the Ph.D. degree in the Electronics Department, Faculty of Electrical Engineering, Djillali Liabes University, Sidi Bel Abbes, Algeria. He is a student member of the "Communication Networks, Architecture, and Multimedia" Laboratory. His principal research interests are in the fields of operating systems, embedded systems, automation systems, systems integration, and the Internet of

Chakib Mustapha Anouar Zouaoui received the Dipl.-Ing. and magister degrees in electrical engineering from Djillali Liabes University, Sidi Bel Abbes, Algeria, and the Ph.D. degree in parallel and distributed computing. He is currently an Assistant Professor with the Department of Electronics, Faculty of Electrical Engineering, Djillali Liabes University, Sidi Bel Abbes, Algeria. As a member of the "Communication Networks, Architecture, and Multimedia" Laboratory, his principal research interests are in the fields of compilation, parallel



Nasreddine Taleb (Senior Member, IEEE) received the Master of Science degree in computer engineering from Boston University, the Elect. Eng. degree from Northeastern University, Boston, and the Ph.D. degree in electrical engineering from Djillali Liabes University, Sidi Bel Abbas, Algeria. He is currently a Professor with the Electronics Department, College of Electrical Engineering, Djillali Liabes University, Sidi Bel Abbas, where he has been teaching since 1990 and where he is also a Senior Research Scientist and the Director of the

“Communication Networks, Architecture, and Multimedia” Laboratory. His principal research interests are in the fields of digital signal and image processing, image analysis, medical and satellite image applications, pattern recognition, and advanced architectures for the implementation of DSP/DIP applications.



Sarah Mazari received the M.S. degree in telecommunications and the Ph.D. degree in electrical engineering from Djillali Liabes University, Sidi Bel Abbas, Algeria. She is currently an Assistant Professor with the Electronic Department, Djillali Liabes University, Sidi Bel Abbas, Algeria. Her research interests include applying computer vision and machine learning techniques in embedded systems to tackle remote sensing problems.



Mohamed El Bahri received the Ph.D. degree from Djillali Liabes University. He is currently a Researcher and an Assistant Professor with the Computer Science Department, where he has been teaching since 2009. His principal research interests are in the fields of image and video processing and analysis, deep learning applied to computer vision, and parallel programming.



Miloud Chikr El Mezouar (Senior Member, IEEE) received the Dipl.-Ing. and magister degrees in electrical engineering from Djillali Liabes University, Sidi Bel Abbas, Algeria, in 1996 and 1999, respectively, and the Ph.D. degree in signal and image processing, under joint supervision, from Djillali Liabes University and from the National Institute of Applied Sciences (INSA), Rennes, France, in 2012. He is currently a Professor with the Department of Electronics, Electrical Engineering Faculty, Djillali Liabes University, Sidi Bel Abbas, Algeria, where

he has been teaching since 2003. He is a member of the “Communication Networks, Architecture, and Multimedia” Laboratory. His principal research interests are in the fields of digital signal and image processing, image analysis, medical and satellite image applications, pattern recognition, electrical and computer engineering, and artificial olfaction systems.